# Introduction to No-SQL Databases




pm jat @ daiict




# Why Relational data model is no more “universal” solution?

Five Issues!

- “**Scale**” (1): Relational Do not scale “horizontally” and “elastically” (Volume Challenge)
- “Normalization” (2) and “Partitioning” do not go well. Query Performances are seriously affected
- “**Strict Schema**” (3) is becoming a problem in “Big Data” (Variety Challenge)
- Programming discomforts (4)
  - Data Objects structurally do not match with relational tables, and this requires lots of “boilerplate codes”
  - ORM tools (like hibernate) do help quite a extent but brings in “complexity” and performance issues!
- “Data loading” is slow (5) – load time validation of schema and other constraints makes loading slow (Velocity Challenge)

# Quiet a few things happened in 2003-2008 [2]

- GFS and Map-Reduce: 2003 and 2004 respectively
- BigTable by Google: 2006
- Hadoop: 2007 at Yahoo, was running on 1000 computers at Yahoo , in 2008 it was behind Yahoo's search engine using 10000 cores.
- Amazon's Dynamo DB: 2007
- HBase (a open source implementation of BigTable): 2008 as Apache project and part of Hadoop
- Facebook's story is said to be bit interesting
  - Started Scaling up MySQL by partitioning the data; but was losing on the front of "JOINS", and "ACID"
  - Started using Hadoop in 2007, and created Hive
  - Around the same time '2008 or so, at Facebook Avinash Lakshman (Creator of Dynamo DB), Prashant Malik created "Cassandra"
- MongoDB, started as 10gen in 2007




# "No SQL" History [1]

- While all such non relational large scale systems were taking birth around
- 2009, some excited folk of database community called for a meet in 2009.
- The term "NoSQL" was a hurray name while folks were figuring out a name for the meet
- This call was for "**open-source, Distributed, non-relational databases**."
- There were presentations in Meet from Voldemort, Cassandra, Dynomite, HBase, Hypertable, CouchDB, and MongoDB.

# The Term “No SQL” [1]

- While trying to understand the term “No-SQL”. Let Relational Databases be referred as “**SQL Databases**”!
- How do we interpret the term “No SQL” and define No SQL database?
- The term can not be interpreted as database “~~systems not having SQL~~”(many no sql systems do provide some variation of SQL, and some, like Cassandra provide almost SQL interface!)
- One most commonly interpretation of No SQL is “Not Only SQL”?
- However this also has a problem?
- This means “Not Only SQL” - does it mean No SQL include SQL (relational) systems too? Not correct (relational are not included in No SQL)




# No-SQL database

- There is no standard definition of No-SQL databases. Text [1]
- Let them be characterized as following:
  - Not Relational
  - Runs on “cluster of computers” – can scale seamlessly well
  - Flexible Schema – can be even schema-less: NoSQL databases operate on either No Schema or have the notion of Flexible Schema (Note: relation databases work on strict schema)!
  - Most of them are “**Aggregation Oriented**” – eases out programming
    - Why Mongo DB is popular
  - Do not implement Referential Integrity
  - Do not implement ACID but have a different notion of consistency “Eventual Consistency”

# Types of NoSQL Databases

- Key-Value
- Document Oriented
- “Column Family” or “Wide Column Databases”
- Graph Databases




# Key Value Databases

- The concept of Key value is not new in Programming world.
- Hopefully, you have used one of following in your programming. Different languages give different names: Maps, Dictionary, Associated Arrays
- Operations are done only on Key, quiet similar to the `Dictionary`
  - Example: Get, Put, and Remove
- For Example:

```
items.put( 313, item_x );
Item a = items.get( 123 );
```

- So forth




# Key Value Databases

- Key Value database uses the concept of “Dictionary” or “Map” in “databases”
- A key value database could very much defined as a “**Persistent Dictionary**” or “**Persistent Map**”. Where
  - value is an “Entity” (Or Aggregate)
  - key is the “primary key” of that entity (Or Aggregate)
- Key-Value databases are also called “Key-Value Stores”
- Make sure that you clearly understand what the meaning of “Key” and “Value” in Key-value databases. This is important.
- These are Aggregation Oriented Database, where key is Key of Aggregate and Value is Aggregate itself?

# Key Value Databases Operations

- In a Key-Value database, the collection of key-value pairs may be called as “Table” or “Collection”
- Example: we have database “empdb”, then read/write operations re performed as following:
- Write: `empdb.employees.put("10011", '{empno:"1001", "name":"Michael", "salary":60000}')`
- Read: `emp1 = empdb.employees.get("10005")`
- Remove: `emp2 = empdb.employees.remove("10001")`

# Example entity (instance) as K-V

- Value in KV DB could be structured or unstructured as byteArray, String CSV, or
- Could be very much structured represented as XML, or JSON
- Point is DBMS DOES NOT SEE it attribute value pairs.
- It sees all of it as a single value
- Only the thing is Key and Values are separated!

Key: "545004"

Value

```
"invoices":
    {
        "invoiceno" : "545004",
        "stockcode" : "47566",
        "description" : "PARTY BUNTING",
        "quantity" : 8,
        "invoicedate" : "2\/25\/2011 12:48",
        "unitprice" : 4.95,
        "customerid" : 13753,
        "country" : "United Kingdom",
        "invdate" : "2011-02-25"
    }
```

# Types of Key-Value Databases

(1) [Vanilla] **Key-Value** Databases or **Key-Value Store**

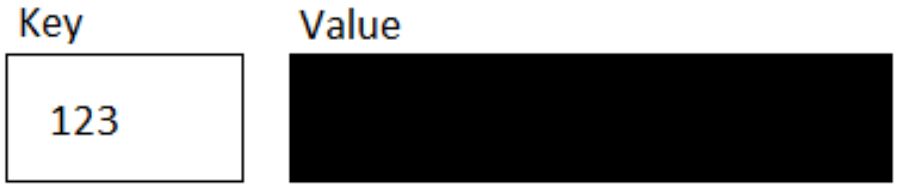
- Value part is opaque
- Operations are based on “KEY” only
  - No operation is possible on any other data from the value part
- Redish, DynamoDB, Riak, RocksDB are examples of

Key: "545004"

Value

```
"invoices":
    {
        "invoiceno" : "545004",
        "stockcode" : "47566",
        "description" : "PARTY BUNTING",
        "quantity" : 8,
        "invoicedate" : "2\/25\/2011 12:48",
        "unitprice" : 4.95,
        "customerid" : 13753,
        "country" : "United Kingdom",
        "invdate" : "2011-02-25"
    }
```

# Key Value Stores - Summarized




- Database here is a "**Persistent Collection**" of "Key-Value pairs"
- Where value is typically a "Data Object" , "Aggregated Data Object"
- Key here, typically could be "Primary Key"
- Value part is transparent and "DBMS does not see and does not process it"
- Values can be stored internal stored in any of data representation format
  - String, Tuple, CLOB, BLOB, XML, JSON.
- **By definition KV databases are schema-less**

# KV database – Pros and Cons

- Pros
  - Efficient queries (very predictable performance) as operations are always based on Key.
  - Easy to distribute across a cluster.
  - No impedance (object-relational) miss-match
- Cons
  - No complex query filters (WHERE(Predicate) part is limited to Keys only)
  - All joins must be done manually through code
  - No foreign key constraints
  - No triggers

# Document Oriented Databases

- Document databases are also built on top of “Key-Value” strategy only
- A KV database does not know anything about the value part, and does not perform any operation on value. It just puts and gets value as a block.
- Whereas in document databases, we specify structural information (partial or full) of value part.
- We can use attributes from value part in SELECT and PREDICATE part
- Note that there is a thin boundary between KV and Document databases!

Document DB

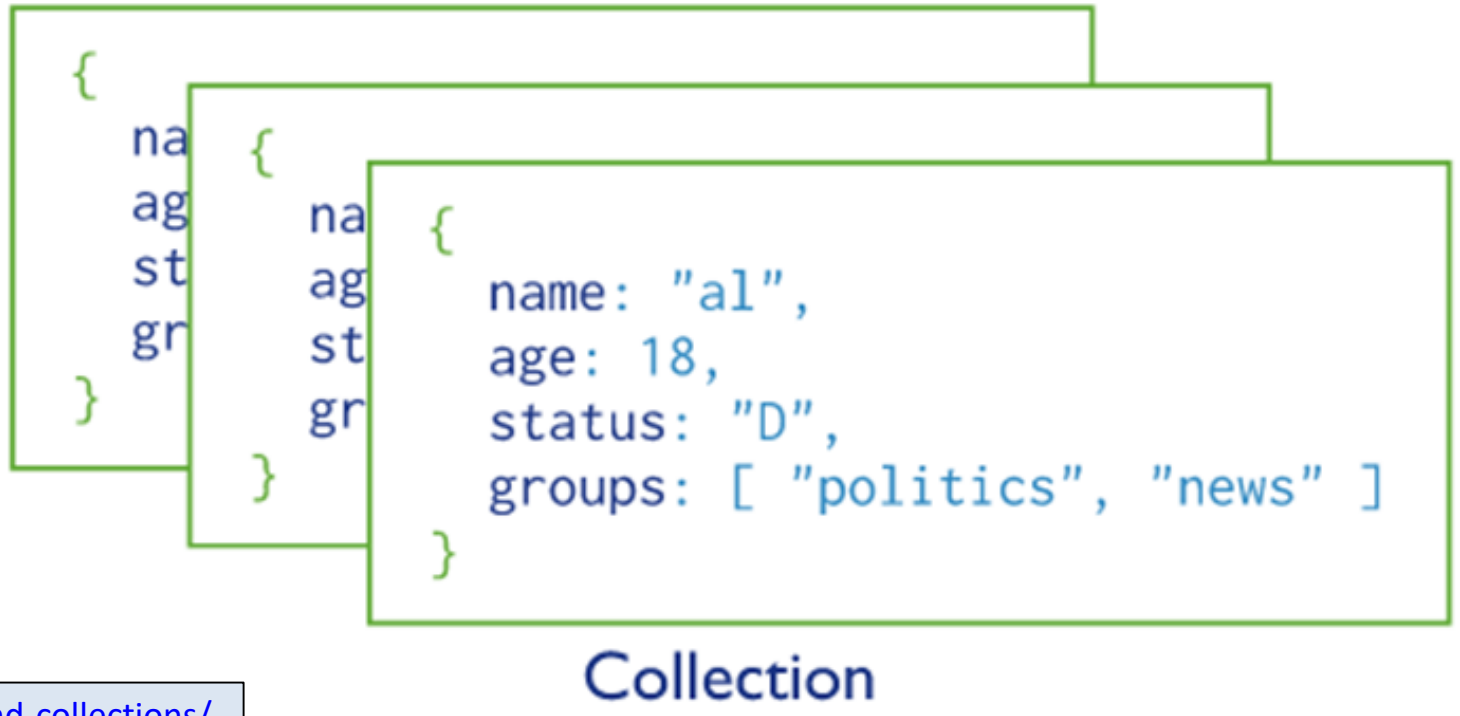
| Key | Value |
| --- | --- |
| 234 | "id": 234,<br>"name": "Mayank",<br>"email": "mayank@gmail.com",<br>"billingAddress": {"city": "Bangalore"} |

Key-Value DB

| Key | Value |
| --- | --- |
| 123 | [redacted] |

# Document Oriented Databases

- Here the term “Document” refers to a record as shown below.
- Document here is a analogues to a row in RDB
- Here is a snapshot from Mongo DB docs.
- MongoDB It calls the collection of documents as “**Collection**” which is analogues to a Table in RDB




Source: https://docs.mongodb.com/manual/core/databases-and-collections/

# Document Oriented Databases

- Here are Mongo DB correspondences with RDB

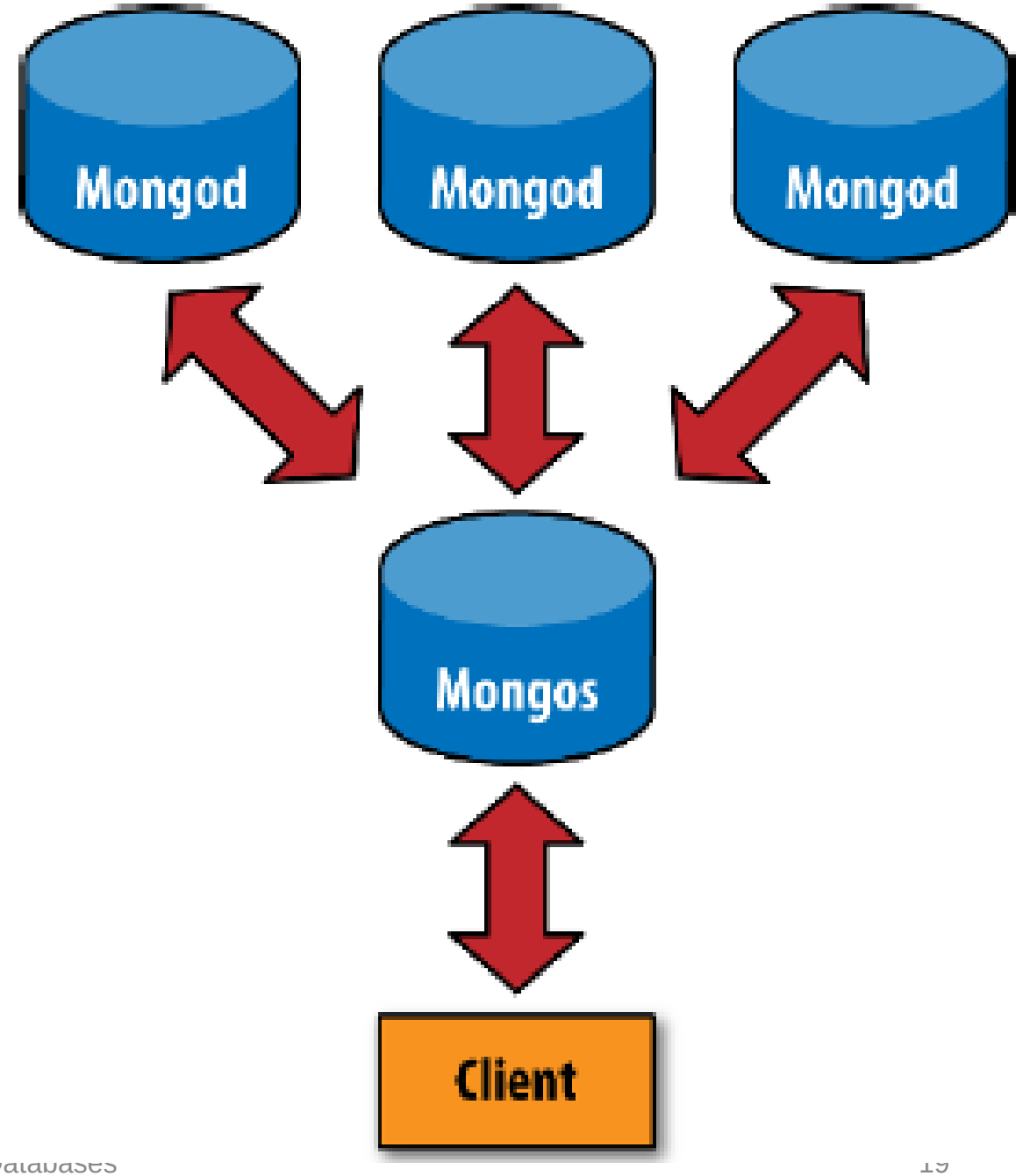
| RDB | Doc DB |
|---|---|
| Database/Schema | Database |
| Table | “Collection” of Documents |
| Row/Tuple | “Document” |
| Row ID | _id |

# Document DB and Schema

- Flexible Schema
  - Mongo DB allows Schema to be “No Schema” to “Full Schema” including Type and Cardinality Constraints
  - We may define set of required attributes and some constraints on them.
  - You can define ID, or Shard Key
  - Schema remain flexible - any extra set of attribute-values.
- DB Design can be “Aggregated” and Normalized” or anything in between
- You can define index on attributes from “Value Part”

# Mongo DB “Sharded”

- Recall modern distributed databases are “partitioned” and “replicated”
- “Sharding” is a special type of “partition” where data records are partitioned “horizontally”
- Each partition has same schema.
- Each partition has a local DB Engine “Wiredtiger” storage engine!






# Popular “Document Databases”

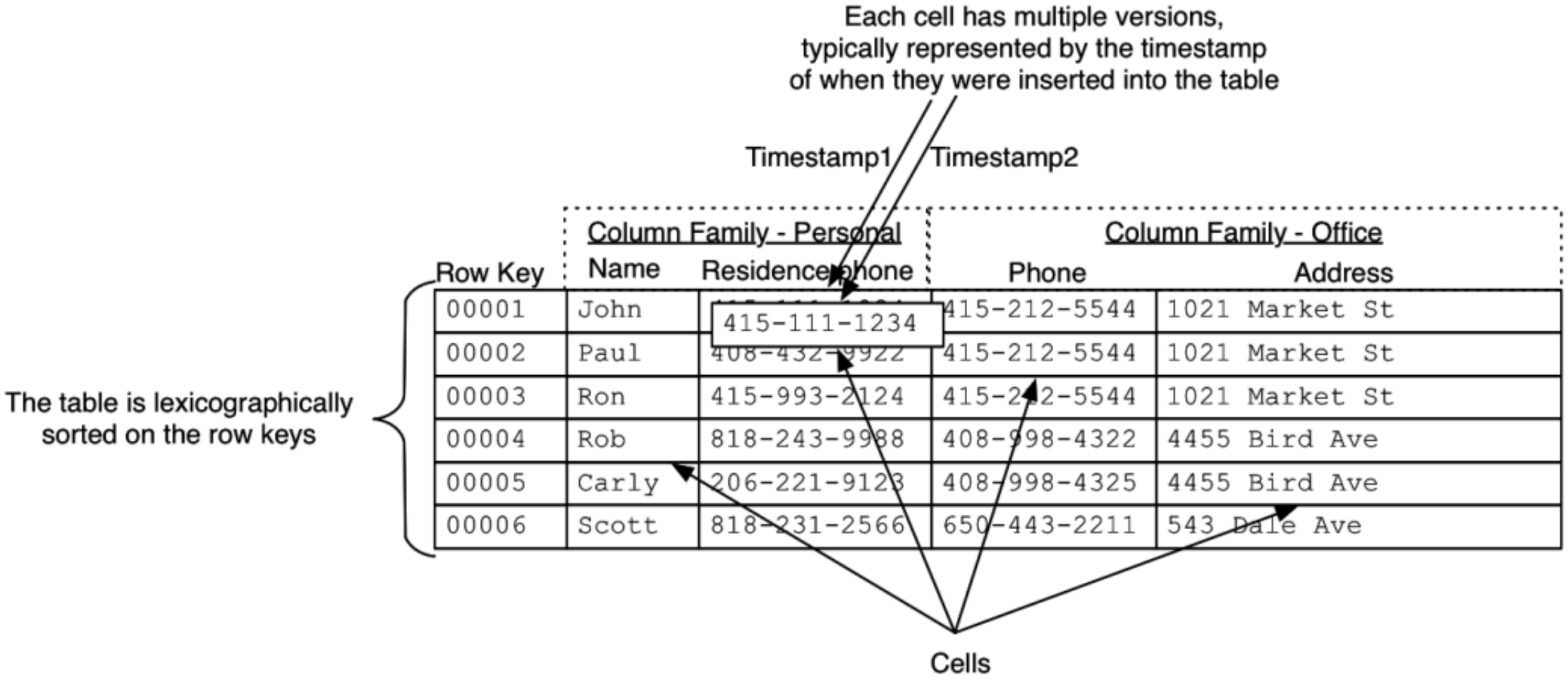
- Popular Document databases are
  - Mongo DB (more towards “Consistency”)
  - Couch DB (more towards “Availability”)
- Different Document databases would have different features in terms of
  - Data Types, and other building blocks
  - API
  - Partitioning, replication, indexing
  - “Consistency Models”




# “Column Family” or “Wide Column” Databases

- “**Column Family**”, First
- Trying to “sound” like relational
  - Has a **Table**, as collection of **Rows**, logically
  - Row is a collection of “unknown set of **columns**” (actually, column-values)
  - Columns are grouped in “**column-families**”
  - A table has a known set of column-families
- So, **why do we call them “Column Family Databases”**?
  - Use the concept of “column family”, that is
  - We do not define the names of columns while creating tables we only define “column families”!

# “Table”, “Row”, “Column Family”, “Column” [5]






# “Column Family”

- Interface: Key-Value
- RowKey as Key and Row as a value
- “RowKey+Col-Family-Name” as Key – Row[Cols-Family-Cols] as value
- “RowKey+Col-Family-Name+Col-Name” as Key – Row[Cols-Family-Cols:Col-Name] as value
- Therefore, it is basically a **multi-level Key-Value Store**
- Analogy: A value in relational model can be said to be two level
  - PK, and PK+Col-Name => employe[‘empno’=101][‘salary’]



# “Column Family” as Key-Value Store

- Multi Level Key-Value store: [row key, column family, column qualifier, timestamp]

00001 → { Personal : { Name : { Timestamp1 : John }, Residence Phone : { Timestamp1 : 415-111-1234 } },
{ Office : { Phone : { Timestamp1 : 415-212-5544 }, Address : { Timestamp1 : 1021 Market St } } }

00001 , Personal → { Name : { Timestamp1 : John }, Residence Phone : { Timestamp1 : 415-111-1234 } }

00001, Personal:Residence Phone → { {Timestamp1 : 415-111-1111 } , { Timestamp2 : 415-111-1234) )

00001, Personal:Residence Phone , Timestamp1 → { 415-111-1111 )

00001, Personal:Residence Phone , Timestamp2 → { 415-111-1234 )




# “Column Family” or “Wide Column” Databases

- Now, why do we call them wide-column databases?
  - Since Names of columns are unbound
  - All rows put together may have thousands of columns, making a row too wide!
- ?
- Examples of “Column Family” or “Wide Column” Databases are:
  - Google’s Bigtable,
  - “HBase” at Hadoop - open soure implementation of BigTable,
  - “Cassandra” at Facebook!




# “Column Family” or “Wide Column” Databases

- Google – BigTable was the first column family databases
- Then “HBase” at Hadoop, “Cassandra” at Facebook followed!
  - HBase is a Hadoop implementation of Big Table only, whereas
  - Cassandra is “Column Family” adaption of “Dynamo DB”, a key-value database
- The BigTable article [2], defines “Big Table” as
  - sparse, distributed, persistent multidimensional sorted “map”.
  - The map is indexed by a row key, column key, and a timestamp; each value in the map is an uninterested array of bytes
- Some of them like Cassandra do have SQL interface, even if it is not a SQL database!

# “Column Family” or “Wide Column” Databases [2]

- “sparse”:
  - Total number of columns in a table could be very large, however individual rows have values only for few of columns
  - This is also reason that these systems are called “**Wide Column**” databases
- “distributed”: partitioned
- “persistent”
- “multidimensional sorted map”.
  - “Map” tells that this is key value store BUT “key can be multidimensional”
  - Rows are kept “Sorted” on key




# “Column Family Databases” and Schema

- While defining a table
  - Row Key is Required
  - Column Family is Required
  - There can be any number of columns in Column Family
- A row required to have a “Row Key” and can have any number of column in any column family
- A row need not to have columns in all column families!



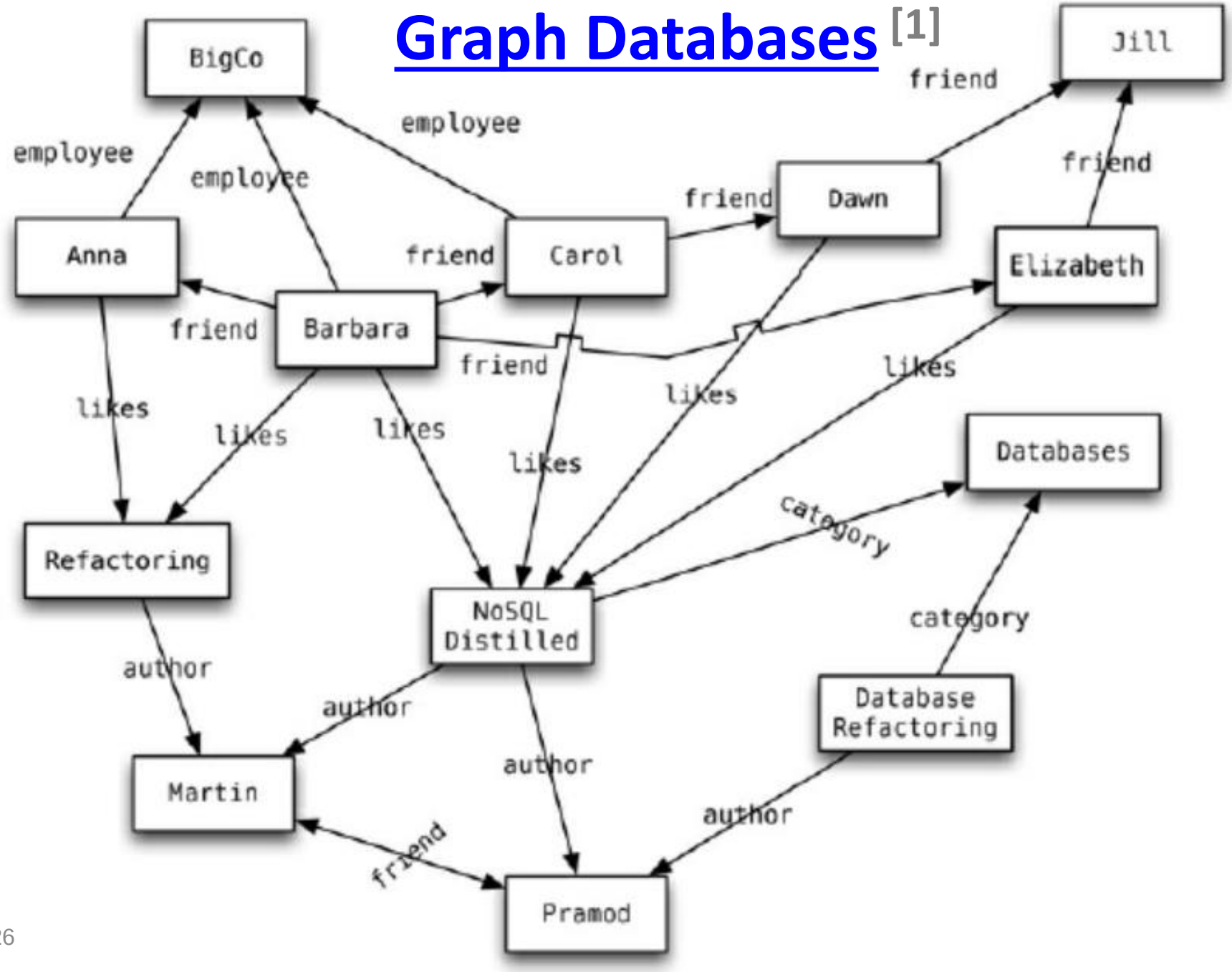

# Why do we three different types of “KV databases”

- Key Value
  - Highly Scalable, Highly Efficient on Key based Access
- Document
  - More friendly with “OOP”, Aggregation Oriented Store.
  - Have no Impedance mismatch problem
  - Tries to have the same functionality of “traditional databases” at scale
  - Access is based on many attributes of an object even attributes of embedded objects
- Wide Column or Column Family
  - Key value beyond mere “KEY”
  - Multi-Dimensional Map
  - We still have a large number of attributes (theoretically millions) still “look up” is amazingly fast and that too at scale.

# Graph Databases

- A node represents an Entity, whereas
- An edge Represents a Relationship
- Operations are performed by Graph Manipulations
- Graph Databases are popularly used in RDF-based Linked Open Data, Social networks, and other information networks.

# Graph Databases [1]



# References and Further Readings


[1] Chapter 1 of book Sadalage, Pramod J., and Martin Fowler. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education, 2013.

[2] Harrison, Guy. *Next Generation Databases: NoSQLand Big Data*. Apress, 2015.

[3] Relational Databases Are Not Designed For Scale
https://www.marklogic.com/blog/relational-databases-scale/

[4] A Brief History of Databases by Stephen Fortune: http://avant.org/project/history-of-databases/

[5] History of Data Modeling: http://graphdatamodeling.com/GraphDataModeling/History.html

[6] Corbellini, Alejandro, et al. "Persisting big-data: The NoSQL landscape." *Information Systems* 63 (2017).

[7] Davoudian, Ali, Liu Chen, and Mengchi Liu. "A survey on NoSQL stores."
*ACM Computing Surveys (CSUR)* 51.2 (2018).

[8] Gajendran, Santhosh Kumar. "A survey on nosql databases." *University of Illinois* (2012).